

Técnicas simples de estimativa de densidade

Alberto Paccanaro
EMap – FGV

www.paccanarolab.org

O material e as imagens nestes slides são de (ou adaptados de):
C. Bishop, Reconhecimento de Padrões e Aprendizado de Máquina, Springer, 2006

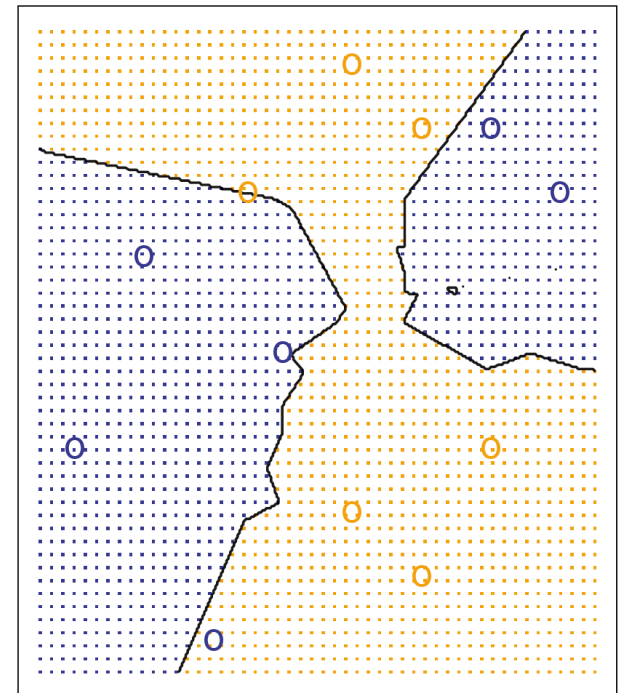
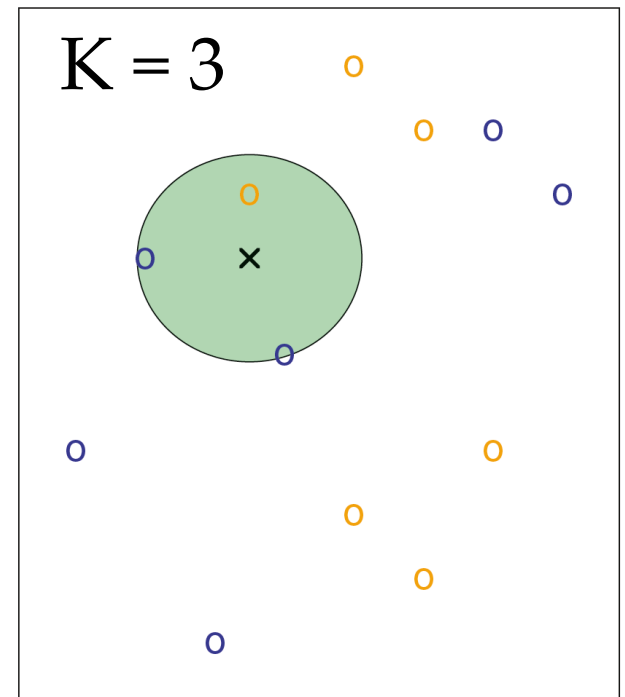
O algoritmo KNN

Conjunto de dados : N datapoints pertencentes a L classes

Escolha: $K > 0$

Para classificar uma nova observação de teste $\mathbf{x}_0$:

- identifique os K pontos vizinhos nos dados de treinamento que estão mais próximos de $\mathbf{x}_0$ -- chame este conjunto de pontos N_0
- para cada classe j , calcule a fração F_j de pontos em N_0 cujos rotulos sejam iguais a j
- classifique a observação de teste $\mathbf{x}_0$ para a classe com o maior F_j



Perguntas para vocês

1. Pera aí... mas afinal, **quais são os parâmetros** que a gente tá aprendendo aqui???
2. **Quais parâmetros controlam a complexidade do modelo? O modelo pode fazer overfit? Como?**
3. **Existe um modelo probabilístico subjacente? Você pode explicar em termos de probabilidades o que estamos fazendo quando aplicamos esse algoritmo?**

...Voltaremos a isso daqui a pouco...

Estimativa de densidade

Queremos obter um modelo $p(\mathbf{x})$ de uma variável aleatória $\mathbf{x}$, dado um conjunto finito $\mathbf{x}_1, \dots, \mathbf{x}_N$ de observações.

Por que isso é importante?

Como eu usaria?

Por exemplo, em **problemas de classificação** .

- Para cada classe, construa $p(\mathbf{x} | C_k)$.
- Em seguida, atribua uma nova entrada $\mathbf{x}_{\text{new}}$ à classe com o maior posterior usando o teorema de Bayes:

$$p(C_k | \mathbf{x}_{\text{new}}) = \frac{p(\mathbf{x}_{\text{new}} | C_k) p(C_k)}{p(\mathbf{x}_{\text{new}})}$$

Por que isso é importante?

Como eu usaria?

Outro exemplo, em **problemas de detecção de outliers** .

Problema: queremos identificar pontos que estão fora do comportamento “normal” de um sistema.

Uma questão importante é que muitas vezes temos muitos datapoints relacionados ao comportamento “normal” do sistema e apenas muito poucos relacionados ao comportamento “anormal” que desejamos identificar.

- construa $p(x)$ usando dados do comportamento “normal” do sistema – este é o seu modelo
- quando uma nova entrada x_{new} calcula sua probabilidade sob o modelo. Se for *muito baixo* , identifique o ponto como “anormal” (outlier).

Estimativa de densidade

Métodos Paramétricos:

1. especificamos uma forma de distribuição. Isso é controlado por alguns parâmetros.
2. Aprendemos os parâmetros, por exemplo, maximizando a probabilidade

Métodos não paramétricos:

1. Evita definir distribuições específicas.
2. Existem parâmetros, mas eles controlam apenas a complexidade do modelo

Métodos paramétricos

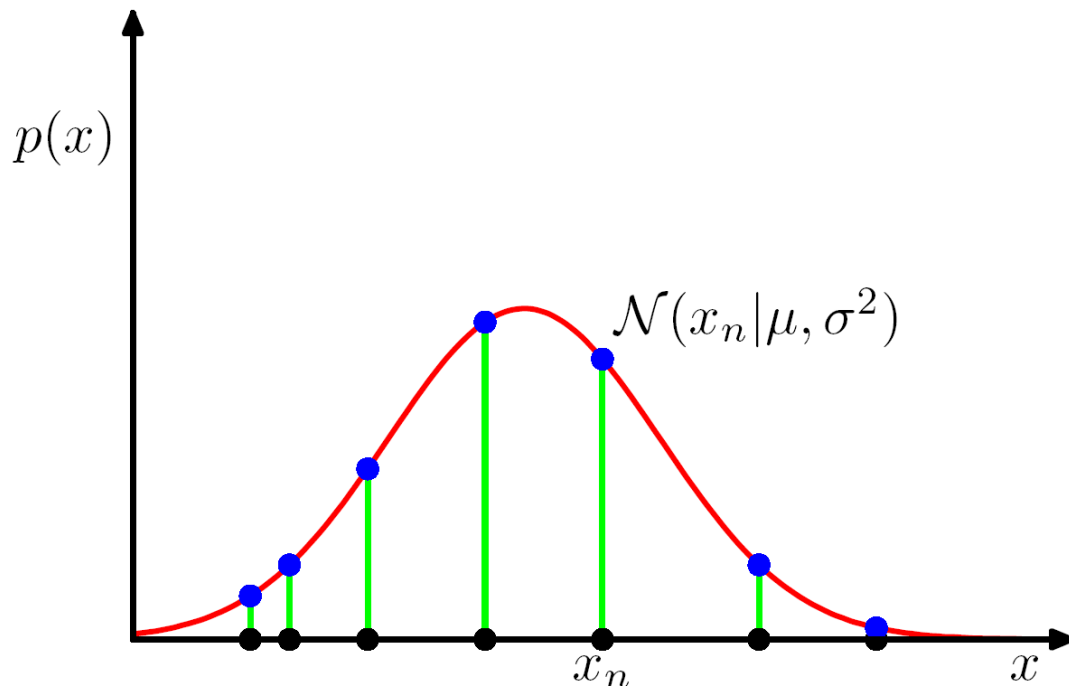
- One Gaussian
- Mistura de Gaussianas

Modelo gaussiano único

Conjunto de dados de observações: $\mathbf{x} = (x_1, \dots, x_N)$

SUPOSIÇÃO : eles foram amostrados independentemente de uma distribuição Gaussiana (iid = *independente e identicamente distribuído*)

$$p(\mathbf{x}|\mu, \sigma^2) = \prod_{n=1}^N \mathcal{N}(x_n|\mu, \sigma^2)$$



Quando visto como uma função de μ e σ^2 , esta é a função de verossimilhança para a gaussiana

Quais valores para μ e σ^2 maximizam a função de verossimilhança?

**Método
paramétrico
simples!**

$$p(\mathbf{x}|\mu, \sigma^2) = \prod_{n=1}^N \mathcal{N}(x_n|\mu, \sigma^2)$$

$$\ln p(\mathbf{x}|\mu, \sigma^2) = -\frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 - \frac{N}{2} \ln \sigma^2 - \frac{N}{2} \ln(2\pi)$$

Maximizando em relação a μ e σ^2 obtemos:

$$\mu_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n$$

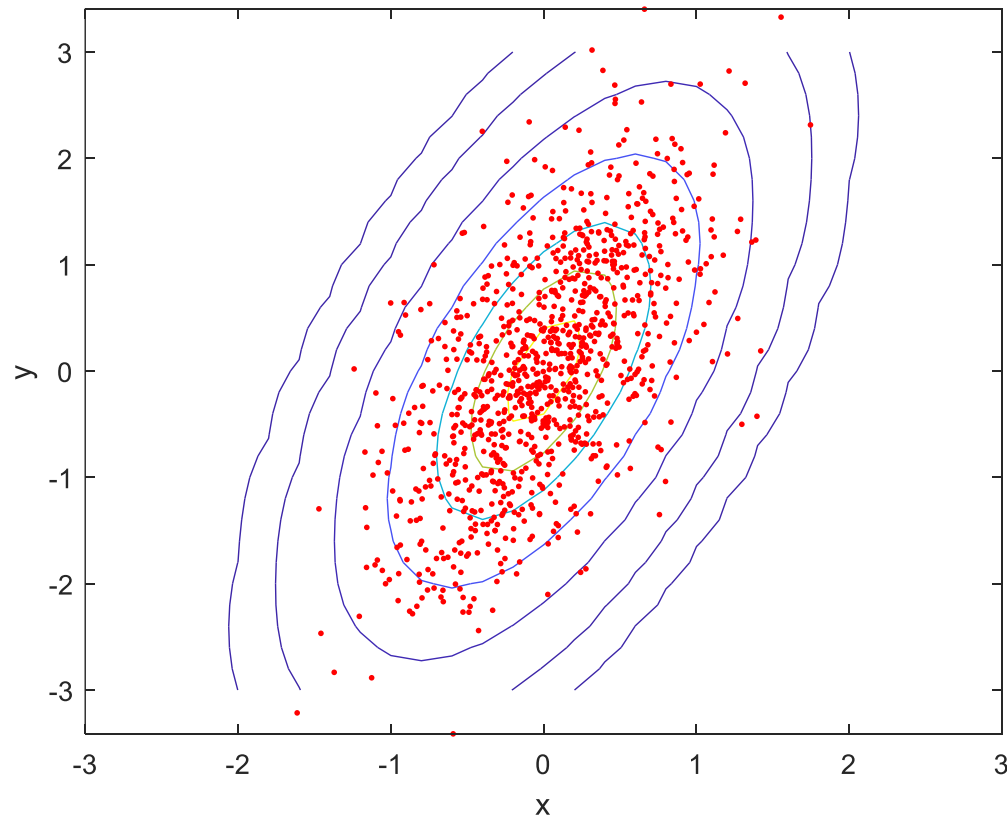
$$\sigma_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \mu_{\text{ML}})^2$$

Portanto, os valores ideais são: **média amostral e variância amostral !**

na prática

Para detecção de outliers:

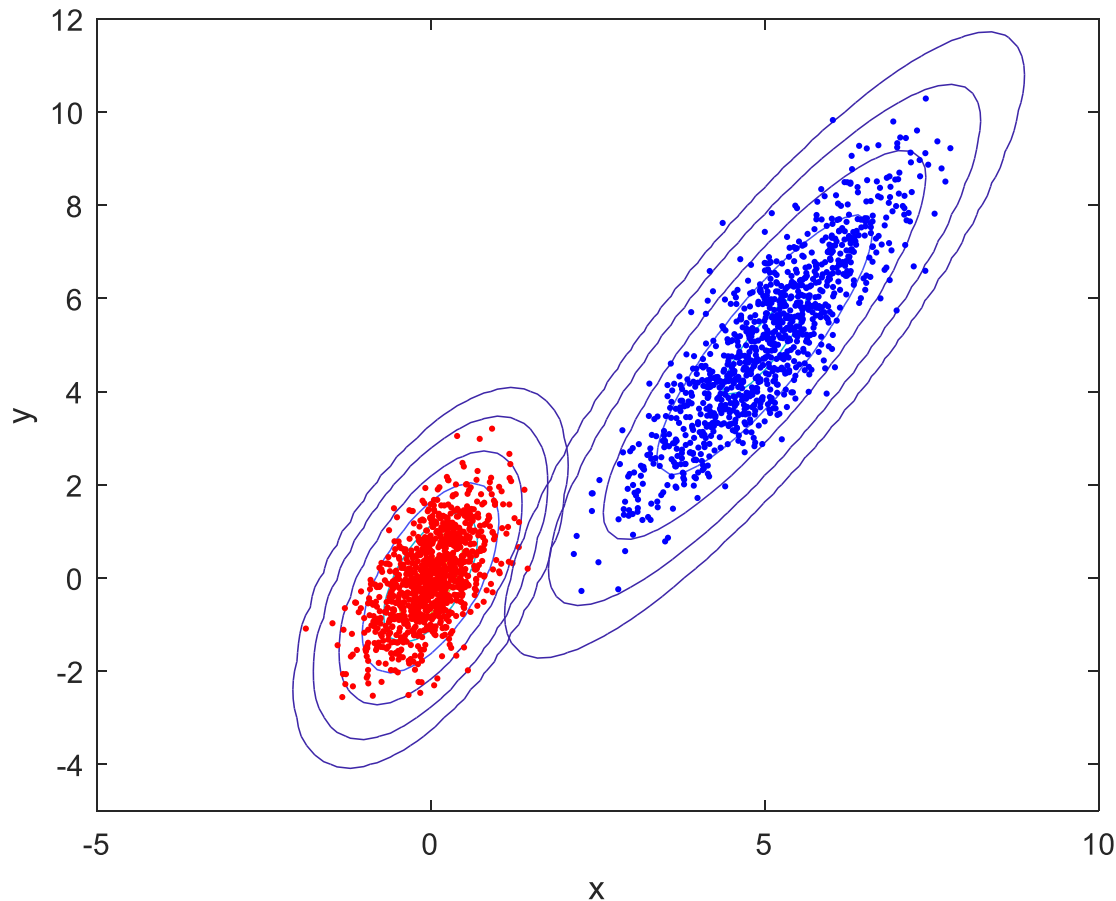
- construa $p(\mathbf{x})$
- para uma nova entrada $\mathbf{x}_{new}$ calcule sua probabilidade sob o modelo.



na prática

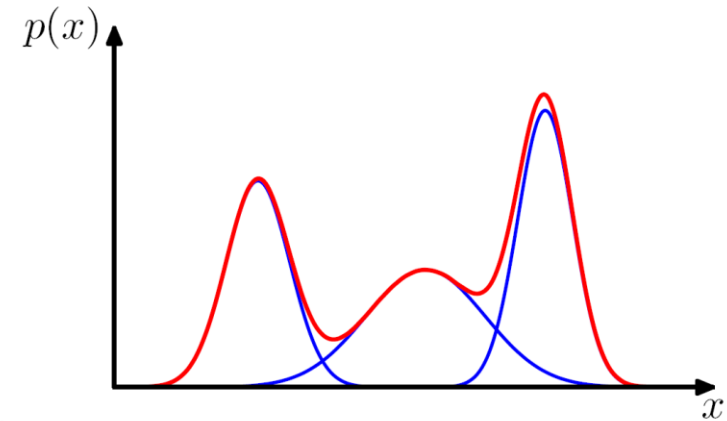
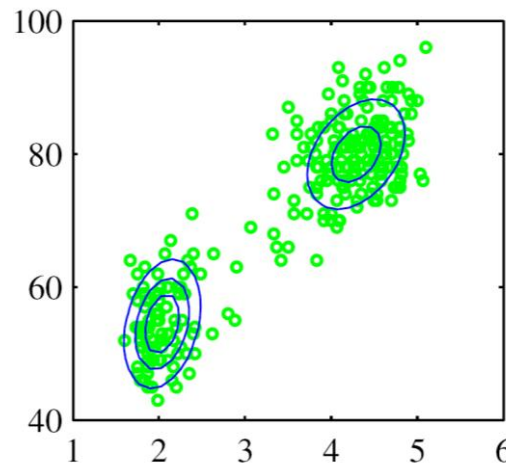
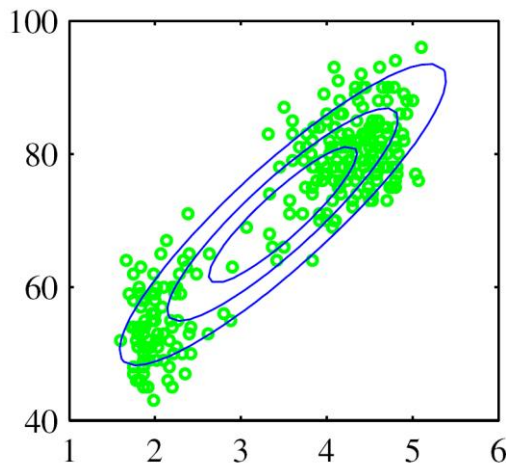
Para classificação:

- para cada classe, construa $p(\mathbf{x}|C_k)$.
- atribua uma nova entrada $\mathbf{x}_{new}$ à classe com o maior posterior usando o teorema de Bayes



Mistura de Gaussianas (MoG)

Uma distribuição gaussiana simples é limitada ao modelar conjuntos de dados reais.



Usando um número suficiente de gaussianas e ajustando suas médias e covariâncias, bem como os coeficientes na combinação linear, as densidades contínuas podem ser aproximadas com precisão arbitrária.

O modelo MoG

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Cada densidade Gaussiana $\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

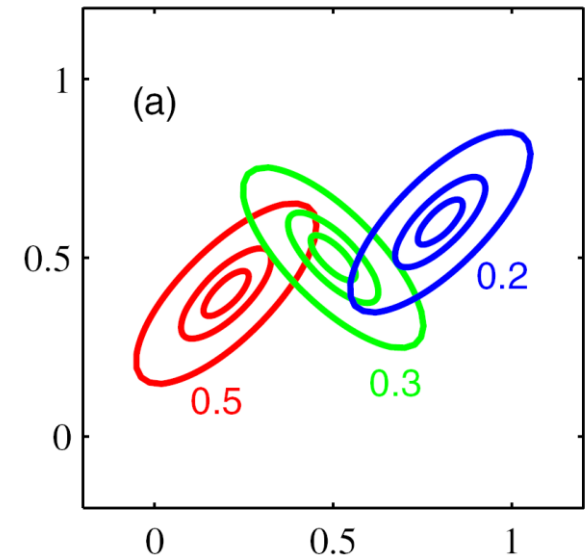
- é um *componente* da mistura
- tem sua própria média $\boldsymbol{\mu}_k$ e covariância $\boldsymbol{\Sigma}_k$.

π_k são os coeficientes de mistura $\sum_{k=1}^K \pi_k = 1 \quad 0 \leq \pi_k \leq 1$

$$p(\mathbf{x}) = \sum_{k=1}^K p(k) p(\mathbf{x} | k) \quad \text{então}$$

$\pi_k = p(k)$ probabilidade a priori de escolher um componente

$\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = p(\mathbf{x} | k)$ probabilidade de $\mathbf{x}$ condicionada em k



Pense nisso como um *modelo generativo*: para gerar um ponto, escolhemos uma gaussiana k com probabilidade π_k e depois geremos um ponto de acordo com $N(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

A melhor maneira de maximizar a função de verossimilhança em relação aos parâmetros é usar o algoritmo EM...

Probabilidades posteriores $p(k|\mathbf{x})$, também conhecidas como *responsabilidades*:

$$\begin{aligned}\gamma_k(\mathbf{x}) &\equiv p(k|\mathbf{x}) \\ &= \frac{p(k)p(\mathbf{x}|k)}{\sum_l p(l)p(\mathbf{x}|l)} \\ &= \frac{\pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_l \pi_l \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)}\end{aligned}$$

O algoritmo Expectation Maximization (EM) para misturas de gaussianas

Ele alterna entre duas etapas:

1. E-step: Calcule as probabilidades posteriores dado o nosso modelo atual – ou seja, quanto acreditamos que cada gaussiana gera cada datapoint (*responsabilidades*).

2. Passo M: Assumindo que os dados foram realmente gerados dessa forma, altere os parâmetros de cada Gaussiana para maximizar a probabilidade de que ela gere os dados pelos quais é atualmente responsável.

1. Initialize the means $\boldsymbol{\mu}_k$, covariances $\boldsymbol{\Sigma}_k$ and mixing coefficients π_k , and evaluate the initial value of the log likelihood.
2. **E step.** Evaluate the responsibilities using the current parameter values

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}.$$

3. **M step.** Re-estimate the parameters using the current responsibilities

$$\begin{aligned}\boldsymbol{\mu}_k^{\text{new}} &= \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n \\ \boldsymbol{\Sigma}_k^{\text{new}} &= \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}}) (\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})^T \\ \pi_k^{\text{new}} &= \frac{N_k}{N}\end{aligned}$$

where

$$N_k = \sum_{n=1}^N \gamma(z_{nk}).$$

4. Evaluate the log likelihood

$$\ln p(\mathbf{X} | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

and check for convergence of either the parameters or the log likelihood. If the convergence criterion is not satisfied return to step 2.

Algumas considerações sobre EM

- Algoritmo iterativo
- O número de gaussianas (K) controla a complexidade do modelo
- Sem taxa de aprendizado (*learning rate*)
- Pode ficar preso em mínimos locais ... *vamos entender isso melhor mais tarde, quando olharmos para as técnicas de gradiente descendente...*

Métodos não paramétricos

- Histogramas
- K-nearest neighbour (😊!)
- Estimativa de densidade com Kernel

Histogramas

Consideramos uma única variável contínua x

- particionar x em compartimentos distintos de largura Δ_i
- conte o número n_i de observações de x caindo no bin i .
- divida pelo número total N de observações e pela largura Δ_i dos bins

$$p_i = \frac{n_i}{N\Delta_i}$$

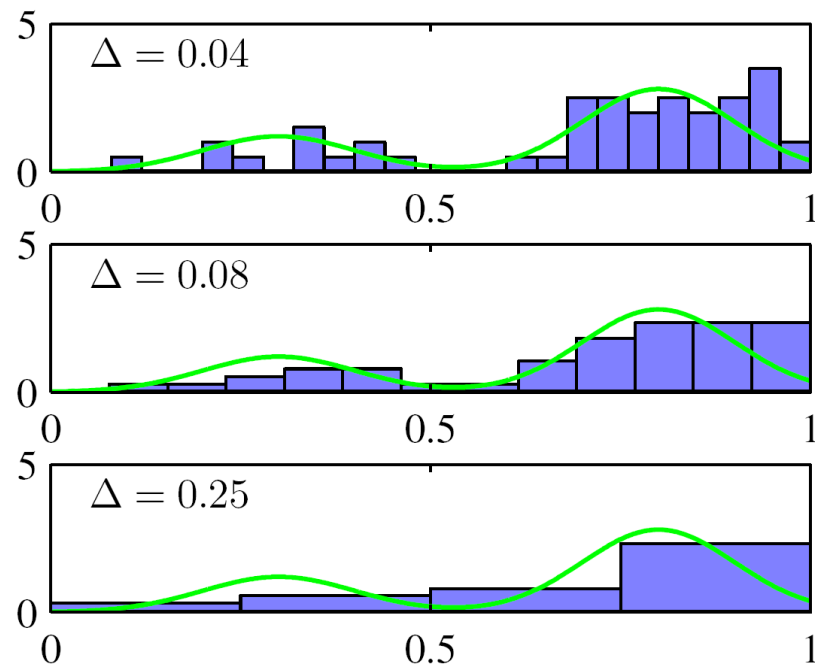
e nós temos

$$\int p(x) dx = 1$$

O conjunto de dados de 50 pontos é gerado a partir da distribuição mostrada pela curva verde.

Os bins podem ter a mesma largura $\Delta_i = \Delta$, ou largura diferente

Aqui, a densidade $p(x)$ é constante ao longo da largura de cada bin.



O que controla a complexidade do modelo?



Vantagem: uma vez que o histograma foi calculado, os dados podem ser descartados (bom se o conjunto de dados for grande)

Desvantagem: escalar com dimensionalidade

Um cálculo “aproximado”...

Extraímos N observações de alguma densidade de probabilidade desconhecida $p(\mathbf{x})$ em algum espaço D -dimensional

→ queremos estimar o valor de $p(\mathbf{x})$

pequena região R , de volume V

K = número de pontos em R

Probabilidade P de estar dentro de R : $P \simeq \frac{K}{N}$

Também temos:

$$P \simeq p(\mathbf{x})V$$

Conseqüentemente:

$$p(\mathbf{x}) = \frac{K}{NV}$$

*Esta eq. nos dá 2 métodos,
dependendo se fixamos K ou V ...*

“simplifiquei”
a matemática...

Caso 1: fixamos K

*obtemos o algoritmo K-NN
para estimativa de densidade*

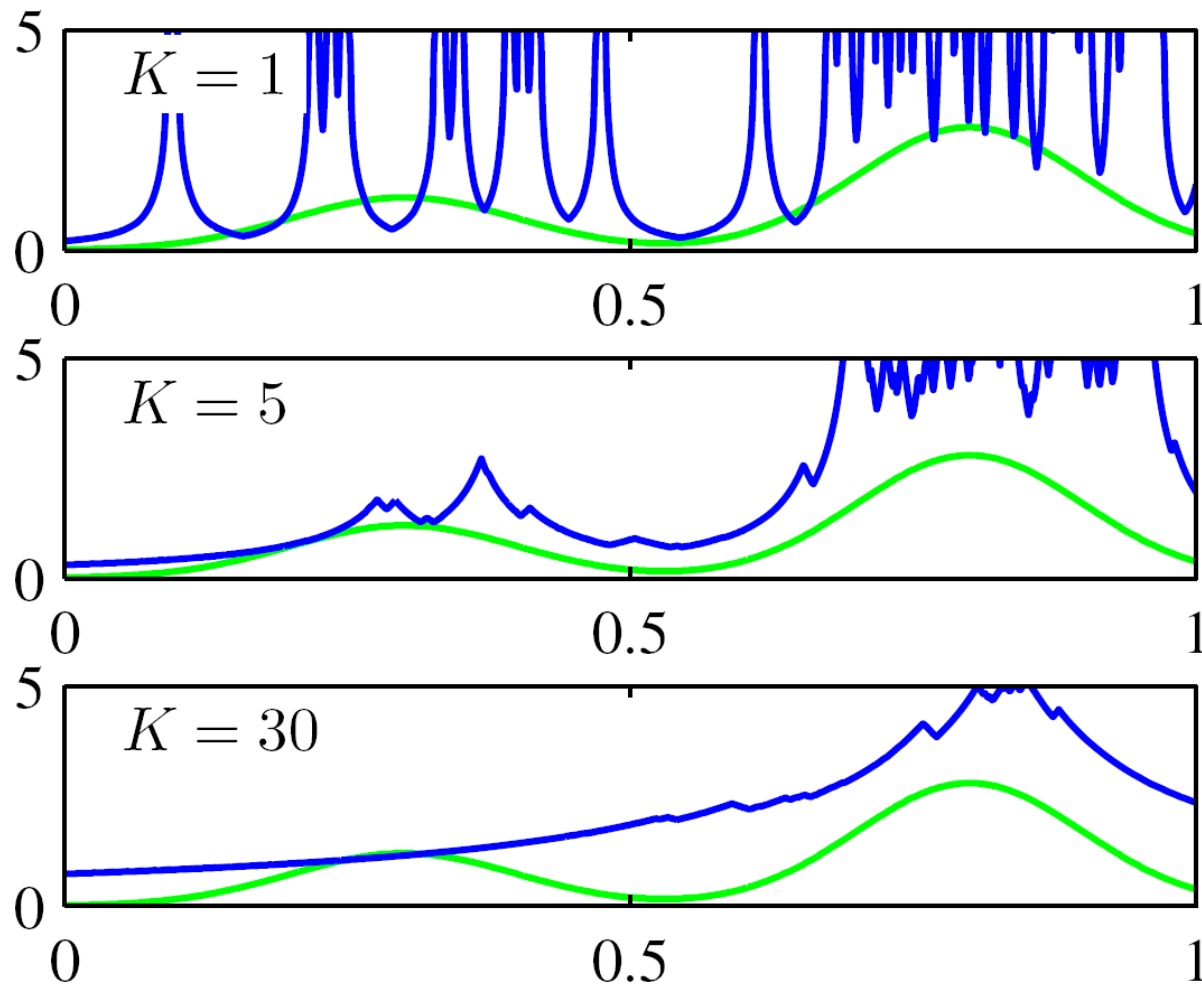
$$p(\mathbf{x}) = \frac{K}{NV}$$

Fixamos o valor de K e usamos os dados para encontrar um valor apropriado para V

- considere uma pequena esfera centrada no ponto $\mathbf{x}$ onde queremos estimar a densidade $p(\mathbf{x})$
- permita que o raio da esfera cresça até que contenha K pontos.
- a estimativa da densidade $p(\mathbf{x})$ é então dada por

$$p(\mathbf{x}) = \frac{K}{NV}$$

com V definido como o volume da esfera resultante



Observe que o parâmetro K governa o grau de suavização...

Então, o que a *classificação* K-nearest-neighbour está fazendo?

Conjunto de dados de N pontos com N_k pontos na classe C_k com $\sum_k N_k = N$

Para classificar um novo ponto $\mathbf{x}$, desenhamos uma esfera centrada em $\mathbf{x}$ contendo K pontos.

V volume da esfera; K_k número de pontos K na classe k

$$p(\mathbf{x}) = \frac{K}{NV} \quad p(\mathbf{x}|C_k) = \frac{K_k}{N_k V} \quad p(C_k) = \frac{N_k}{N}$$

Temos todas as peças para aplicar o teorema de Bayes:

$$p(C_k|\mathbf{x}) = \frac{p(\mathbf{x}|C_k)p(C_k)}{p(\mathbf{x})} = \frac{K_k}{K}$$



De volta às nossas perguntas originais...

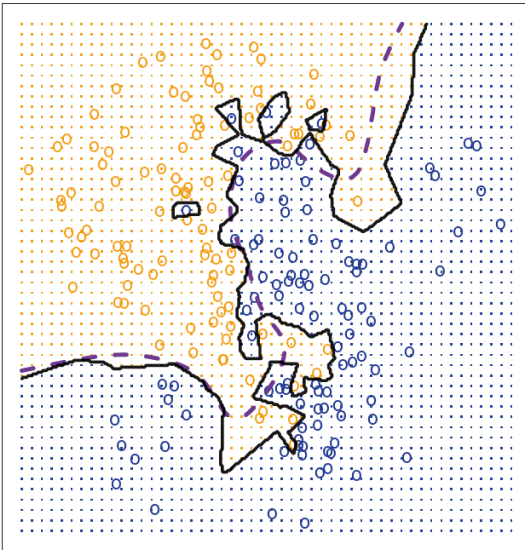
1. Existe um modelo probabilístico subjacente? Você pode explicar em termos de probabilidades o que estamos fazendo quando aplicamos esse algoritmo?

K-NN está minimizando a probabilidade de classificação incorreta, atribuindo o ponto $\mathbf{x}$ à classe com a maior probabilidade posterior.

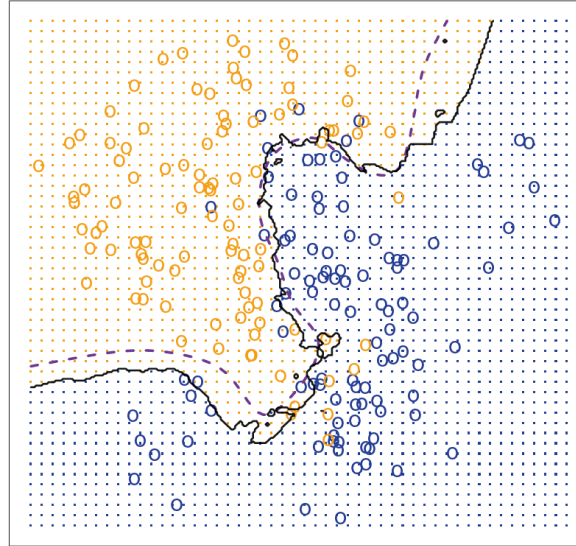
Nota: está estimando $p(\mathbf{x})$ de uma maneira muito simples e não paramétrica.

2. Qual parâmetro controla a complexidade do modelo? O modelo pode overfit? Como?

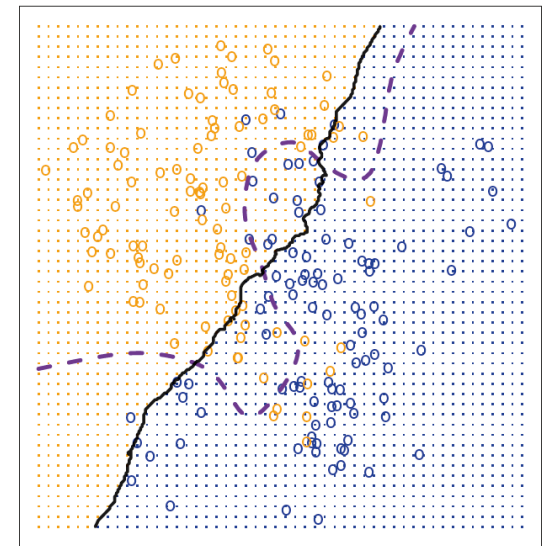
KNN: $K=1$



KNN: $K=10$



KNN: $K=100$



Quando $K = 1$, o limite de decisão é excessivamente flexível. À medida que K cresce, o método se torna menos flexível e produz um limite de decisão próximo do linear.

Caso 2: fixamos V

$$p(\mathbf{x}) = \frac{K}{NV}$$

obtemos a estimativa de densidade com Kernel...

- Região R : um pequeno hipercubo centrado em $\mathbf{x}$
- para contar o número K de pontos caindo em R , defina:

$$k(\mathbf{u}) = \begin{cases} 1, & |u_i| \leq 1/2, \\ 0, & \text{otherwise} \end{cases} \quad i = 1, \dots, D \quad (\text{um cubo unitário centrado na origem})$$

$$k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right) = \begin{cases} 1 & \text{se o datapoint } \mathbf{x}_n \text{ está dentro de um cubo de} \\ & \text{lado } h \text{ centrado em } \mathbf{x} \\ 0 & \text{caso contrário} \end{cases}$$

- número total de datapoints dentro deste cubo:

$$K = \sum_{n=1}^N k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right)$$

$$p(\mathbf{x}) = \frac{K}{NV}$$

- O volume de um hipercubo de lado h em D dimensões é
 $V = h^D$

- A densidade estimada em $\mathbf{x}$ é:

$$p(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h^D} k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right)$$

(soma sobre N cubos centrados nos N datapoints $\mathbf{x}_n$).

- $k(\mathbf{u})$ é uma *função kernel* (neste contexto, também é chamada *Parzen window*).
- O modelo de densidade é chamado de **estimador de densidade de kernel** (ou estimador de *Parzen*).

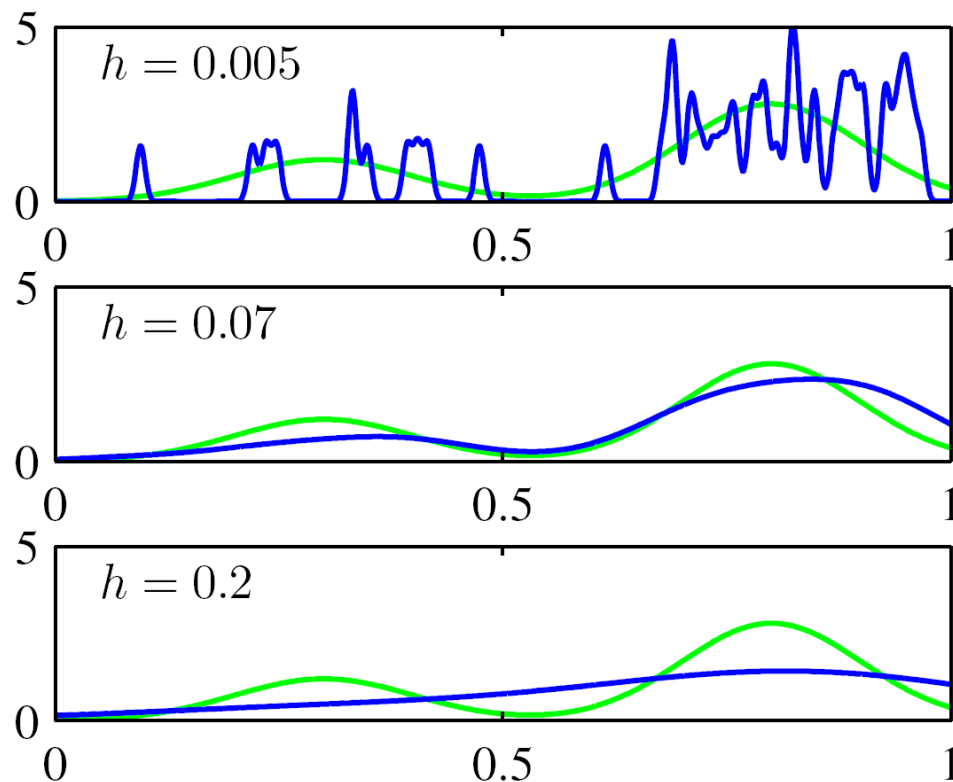
- Podemos usar uma função kernel mais suave
- Por exemplo, usando um gaussiano:

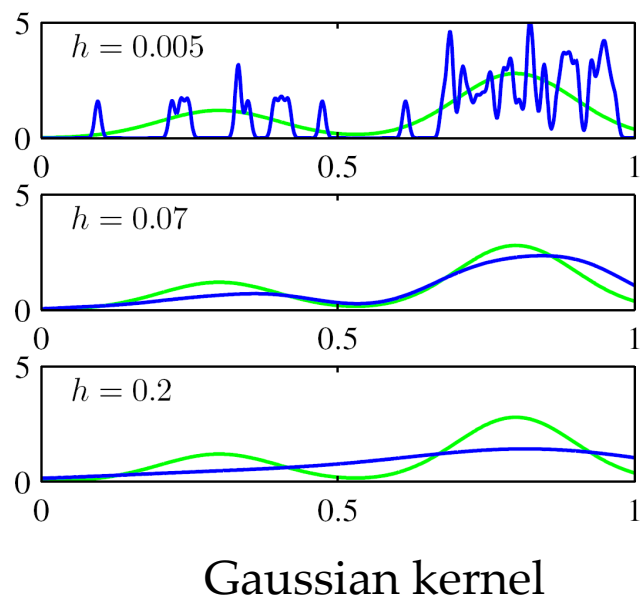
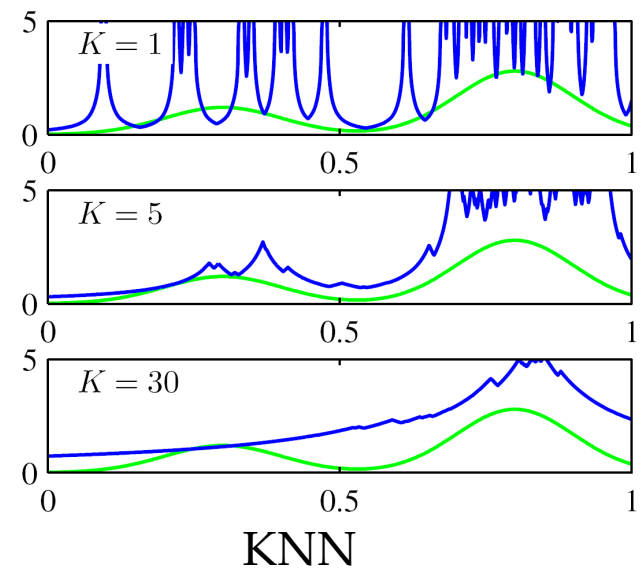
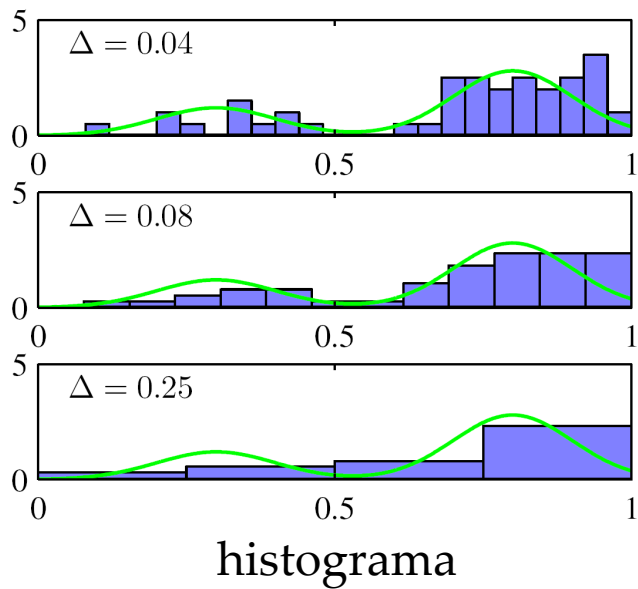
$$p(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{(2\pi h^2)^{1/2}} \exp \left\{ -\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{2h^2} \right\}$$

O modelo de densidade é obtido por:

- colocando uma gaussiana sobre cada datapoint
- somando as contribuições sobre todo o conjunto de dados,
- dividindo por N para normalizar.

O que controla a complexidade do modelo? h





Naïve Bayes: um método não paramétrico de classificação

Classificação de duas classes, de um vetor $\mathbf{x} = (x_1, \dots, x_n)$ em C_0, C_1

$$P(C_k | \mathbf{x}) = \frac{p(\mathbf{x} | C_k) \cdot P(C_k)}{p(\mathbf{x})}$$

Ideia: faça a suposição simplificada de que os valores de atributo são condicionalmente independentes dados os valores de destino

$$p(\mathbf{x}|C_i) = p(x_1, x_2, \dots, x_n|C_i)$$

$$\approx p(x_1|C_i) \cdot p(x_2|C_i) \cdot \dots p(x_n|C_i)$$

$$= \prod_j p(x_j|C_i)$$

$$\begin{aligned} P(a, b, c) &= P(a | b, c)P(b, c) \\ P(b, c) &= P(b | c)P(c) \\ P(a, b, c) &= P(a | b, c)P(b | c)P(c) \end{aligned}$$

A aprendizagem em Naive Bayes consiste em estimar os vários $P(C_i)$ e os vários $p(x_j|C_i)$ usando a equação acima com base em suas frequências sobre os dados de treinamento.

Importante

- $p(x_i|C_k)$ pode ser modelado por uma distribuição paramétrica (por exemplo, Gaussiana)
- Funciona com **missing data!!!**
- Os resultados são facilmente **interpretáveis !!!**
- Para muitos problemas, funciona extremamente bem 😊